

Description

Babel is a system that allows a language specialist to review (LQA) translations in a graphical context that pairs screenshots in english next to another language. It allows for better disambiguation.

There are 3 basic parts to this system:

1. Backend (Server) Java AppEngine application, uses DataStore to store data, and GCS for images. Images are only handled during the upload process by a producer, where the images are copied cloud to cloud.
2. Frontend (Client) Google Dart application, deployed also in the backend to serve the pages, but executed locally in the user's browser. Frontend accesses GCS images directly through secured signed urls.
3. Android Screenshot Generator, Java UiAutomator plugin that allows producers (content providers) to write scripts in the Java language to navigate an Android app and to generate screenshots as it goes. Content (images and data) are then downloaded to the producer's computer. Images are copied to GCS via gsutil (a python application developed by Google.) Data is uploaded to Babel through the frontend and it causes records to be created or updated, so that they can be viewed by the reviewer in the frontend.

System operation

- Logon: User opens a browser and opens Babel url. The first time the user is prompted to select an account to use and then asked for credentials. The user may later logout, to use a different account, or choose to stay logged on and the system will re-use the logon credentials from the browser if so configured.
- Hand shake: The front end requests information from the server to identify the user and to see if its authorized and what role the user has.
- Round selection: user selects a round and data is fetched from the server. Data is used to populate the user's view.
- Depending on the role the user would perform any necessary operations for various tasks.
- Background tasks: there are two types:
 - Upload: These should only exist when producers are working uploading new screenshots.
 - Asynchronous Command: There are only a few of these, usually dealing.

Client/Server connectivity is done using Google Endpoints for all API calls. The user must be authenticated for all calls. The system uses RBAC (Role Based Authorization) for all calls. Some API calls have additional authorization based on the resource type. We use two resources to provide authorization, Application and Locale.

For more details please see architectural document.

AppEngine Endpoints:

http://apis-explorer.appspot.com/apis-explorer/?base=https://gloc-babel.appspot.com/_ah/api#p/antaeus/v1/

What could go wrong/What has been observed to go wrong

When issues happen refreshing the browser usually helps in several situations.

System Access

- OAuth2, we do not handle anything related to these, see OAuth2 troubleshooting guides.
- "Unauthorized user, please contact a Babel administrator to obtain credentials" This indicates that the user is *not* in Morpheus or in the UserOverride table. When this happens the easiest thing to do at the moment is to add the user to the table.

Images not showing

- This can be due to GCS file copy lag, or to appengine not having run the midnight cron job. To remedy this from appengine http://apis-explorer.appspot.com/apis-explorer/?base=https://gloc-babel.appspot.com/_ah/api#p/antaeus/v1/antaeus.api.cronMidnight run this endpoint.

Infrastructure failures

We have made several attempt to minimize dependencies to make the system more robust and resilient. We depend on the following: OAuth2, AppEngine, GCS. We also depend (but these shouldn't cause service interruption) Channels

- GCS occasionally has lag which is experienced when copying a file. The file will not show until after a few minutes. The upload process has been upgraded to deal with this, but it may still cause issues for producers where they have to wait and later trigger generation of signed urls manually, similar to running cronMidnight endpoint. We have not observed any other issues with GCS, in other words, once the files show up, they remain available. With one exception: Signed Url generation causes a file to be created in GCS, this file is then used by the client so that it can view the images, it contains a list of signed urls. If this file is unavailable the user will have to click on refresh at a later time and this can be a few minutes.
- OAuth2, this failed once before. If this happens, the service will be interrupted for the duration. There is nothing to be done here, since the system requires security.
- AppEngine, there have been several failures here on several occasions. It will most likely happen again. Sometimes shutting down the backend instances will help. Check to make sure if you do this that there are no background tasks running, or the shutdown will fail. Depending on the kind of background task running, it may not be ok to stop some unless you can redo that work.
- Channels API is used for several operations and so that the server can talk to the client. Channels API regularly fails and we do not show the users these errors, but they are always logged in the server. When Channels fails the user would not see, progress bar,

some error notifications from the server, record updates from other users, so that for instance if two users are working on the same record (screenshot) they would not know. However, that use case scenario was not requested and it's a kind of nice to have feature, but users should be instructed not to work on the same data at the same time.

Data Access

- Data reads/writes has been steadily reliable. Concurrency also has been reliable.
- Caching is done using memcache. Caching has failed sometimes, so the system robustness was increased by allowing the data fetch process to bypass caching in the event of failure. This would cause a slowdown when getting Round data (DataComposite) during round selection. The effect would be a delay of anywhere from 2 to 5 seconds. This is a sporadic issue.

System Errors

- When errors occur, data is logged and the frontend is notified and the user would be able to see the error in the log or as a message box.
- There are two main sources of errors: backend and frontend, they are treated about the same.
- Backend errors are reported via endpoint call completion for each API call and via Channels. They are complementary and not duplication of response.

Maintenance

Notification

Contact Kevin Guerra (guerrak@google.com) for any questions. Eventually we expect to have a support group and more developers.

Access Granting Policy and Process

For Web access:

You need:

1. An Idap you can use to log into Google via OAuth2. This can be your gmail address, or another email address that can be authenticated.
2. To be granted access to Babel in either two ways:
 - a. Morpheus, if you are authorized in Morpheus as an LPM or language specialist, and other authorized roles, you're already in Babel. Morpheus also determines which locales (depending on the role) the user is allowed to view or work with.
 - b. Babel Administration. This is for cases where the user is not in Morpheus or we need to alter the authorization roles or locales allowed.

NOTE: Currently Babel Administration is done in the appengine console, using what we call "User Override" table, where you enter a user, role and authorized locales, by simply adding a new record. This will change once we have the user admin module, from which an administrator, manager or LPM will be able to grant access in various ways.

For API access:

API Access is not needed directly. The Dart client (Babel frontend) 'talks' to the backend via Google endpoints, after proper authentication via OAuth2 is done.

Log analysis

Babel logs most relevant information into the appengine log. The system will log for normal operation information that identifies the type of operation, the user's ldap, and relevant steps. It tries not to be too verbose, but there's always a tradeoff.

In the event of a failure, the system will log more detailed information such as parameters, and full exception stack. The system uses the null object pattern so as to avoid most situations about nulls.

Most input is validated at the server. Currently the library we use to validate java class objects is broken and it's disabled until we find a solution. It should be enabled so that we can validate class objects as well. At the moment we do not transfer class objects from client to server so it's more or less ok until we have enough time solve.

Design Document

- System Design document:
https://docs.google.com/document/d/16XAbCP9ZbvYi-_06g9aJBrZD3-2UQIZ4nOrPU9XD0Rs/edit
- Demo: https://docs.google.com/presentation/d/1-AyTj0Fudk0VNRkHWS1V0Awv5TtNAVdA182h1TiSXkg/edit#slide=id.g4d0007b53_0102

Access Pointer

- Production: <https://gloc-babel.appspot.com>
- Sandbox: <https://gloc-babel.appspot.com>
- Endpoints: http://apis-explorer.appspot.com/apis-explorer/?base=https://gloc-babel.appspot.com/_ah/api#p/antaeus/v1/

Source code location

git: <https://code.google.com/a/google.com/p/l10n-avik/>

backend:

<https://code.google.com/a/google.com/p/l10n-avik/source/browse/Antaeus>

frontend: <https://code.google.com/a/google.com/p/l10n-avik/source/browse/#git%2FAntaeus%2Fdart%2Fbabel>

Android Screenshot Generator: <https://code.google.com/a/google.com/p/l10n-avik/source/browse/AndroidScreenGenerator>

Monitoring

- Babel/Pingdom/Uptime: <https://share.pingdom.com/banners/e4c1db2e>
- Babel/Pingdom/Latency: <https://share.pingdom.com/banners/5041b7ea>
- Babel/StackDriver/Latency: <https://public.google.stackdriver.com/public/chart/vlYAVe4EGHekQN8w>
- You can also monitor Babel using the AppEngine interface.